CASE STUDY 5: USER ACCOUNTS, EMAIL VERIFICATION AND CONTACT DETAILS

2520030123

POLISETTY SREEHARSHA

Q1. Identification of Invariants

The following business invariants must be maintained in the user account management system:

1. Email Address Validity: Every user must register with a properly formatted email address. This prevents incorrect email addresses and delivery failures.
2. Email Verification: A user should be marked as verified only after successfully verifying their email. Otherwise, unverified users may access restricted services.
3. Verification Token Expiry: Verification tokens must have an expiry time. Expired tokens should not be accepted to prevent unauthorized verification.
4. Phone Number Representation: Phone numbers should be stored in a standard format, including the country code. This avoids confusion and makes communication easier.
5. Postal Address Completeness: A postal address must contain required details such as street, city, postal code and country. Incomplete addresses may cause delivery problems.
6. User Account Status: Suspended or deleted users must not be allowed to log in. This helps maintain account security.
7. Email Address Uniqueness: Every user must have a unique email address, regardless of uppercase or lowercase letters. This prevents duplicate accounts.

Q2. Entities and Value Objects

The domain concepts can be classified into two types: Entities and Value Objects.

Concept	Classification	Justification
User Account	Entity	It has a unique identity and exists throughout its lifecycle.
User Identifier	Value Object	It represents the unique ID of a user and is compared by its value.
Email Address	Value Object	It is identified by its value and does not have a separate identity.
Phone Number	Value Object	It represents contact information and is compared by its value.
Postal Address	Value Object	It contains address details and is compared based on its contents.

Verification Token	Value Object	It contains a token value and expiry time and can be replaced when expired.

Conclusion: Entities have a unique identity, whereas Value Objects are identified by their values. Value Objects are generally immutable, which helps prevent invalid changes.

Q3. Making Illegal States Unrepresentable

Problems with the existing design:

Using primitive data types, nullable fields and Boolean flags can create several problems:

- Invalid email addresses may be stored.
- emailVerified may be true even when verifiedAt is null.
- Expired verification tokens may be accepted.
- Phone numbers may be stored in different formats.
- Postal addresses may contain incomplete information.
- Suspended users may still be allowed to log in.
- Multiple Boolean flags may create confusing or contradictory states.

Proposed improved object-oriented design:

We can create separate classes for different domain concepts instead of storing everything as primitive data types.

1. Suitable domain-specific types:

- EmailAddress – validates and stores email addresses.
- PhoneNumber – stores phone numbers in a standard format.
- PostalAddress – stores complete address details.
- UserId – represents the unique user ID.
- VerificationToken – stores the token and its expiry time.
- UserAccount – manages user details and account operations.

2. Validation Rules:

- Email must have a valid format.
- Phone number must follow a standard format.
- Postal address must contain all required fields.
- Verification token must not be expired.
- Suspended and deleted accounts must not allow login.

3. Representation of Email Verification States:

Instead of using Boolean flags, we can use an enum:

```
enum EmailVerificationStatus {
    PENDING,
```

```
    VERIFIED
}
```

This avoids contradictory verification states.

4. Representation of Optional Contact Information:

Phone numbers can be represented using Optional<PhoneNumber> so that users can register without providing a phone number.

5. Java Class Definitions:

```
class EmailAddress {
    private final String value;

    public EmailAddress(String value) {
        if (value == null || !value.matches(
            "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")) {
            throw new IllegalArgumentException("Invalid email");
        }
        this.value = value.toLowerCase();
    }

    public String getValue() {
        return value;
    }
}

class VerificationToken {
    private final String token;
    private final LocalDateTime expiryTime;

    public VerificationToken(String token,
                             LocalDateTime expiryTime) {
        this.token = token;
        this.expiryTime = expiryTime;
    }

    public boolean isValid() {
        return LocalDateTime.now().isBefore(expiryTime);
    }
}

class UserAccount {
    private EmailAddress email;
    private EmailVerificationStatus verificationStatus;
    private Optional<PhoneNumber> phone;
    private UserStatus status;

    public void verifyEmail() {
        verificationStatus = EmailVerificationStatus.VERIFIED;
    }

    public boolean canLogin() {
        return status == UserStatus.ACTIVE &&
               verificationStatus == EmailVerificationStatus.VERIFIED;
    }
}
```

```
class PhoneNumber {
    private final String number;
    public PhoneNumber(String number) {
        if (number == null ||
            !number.matches("\\+?[0-9]{10,15}")) {
            throw new IllegalArgumentException(
                "Invalid phone number");
        }
        this.number = number;
    }
    public String getNumber() {
        return number;
    }
}
enum UserStatus {
    ACTIVE,
    SUSPENDED,
    DELETED
}
```

Here, LocalDateTime, PhoneNumber and UserStatus are required Java types or domain types that should be defined separately.

Conclusion: Using separate classes, validation rules and enums helps prevent invalid data and makes the application more secure and reliable.

Q4. Email Verification and Account Lifecycle (6 Marks)

- Information maintained during registration:
  The system should store User ID, Name, Email address, Password (in hashed form), Phone number (optional), Registration date, Verification status and Account status.
- Verification Token Generation and Validation:
  Generate a unique and secure random token during registration. Send the token to the user's registered email address. Store the token along with its expiry time. When the user clicks the verification link, check whether the token is correct and valid.
- Handling Expired Verification Tokens:
  If the token has expired, the system should reject it and allow the user to request a new verification token.
- When a Verified User Changes Their Email
  Update the email address only after validating it. Change the verification status to PENDING. Generate a new verification token. Send a verification link to the new email address. Mark the new email as verified only after successful verification.

- Preventing Suspended and Deleted Accounts from Logging In:
  The login system should check the account status before allowing access. Suspended or deleted users should be denied login.
- Preventing Contradictory Verification States:
  Instead of using multiple Boolean fields, use an enum such as PENDING and VERIFIED. Store verification details only when applicable. This ensures that the account does not enter an invalid verification state.

Q5. Cross-Aggregate Rules and Design Evaluation

- Is email uniqueness an invariant of an individual User object?
  No. Email uniqueness is not an invariant of an individual User object because it depends on all registered users in the system. Two different users should not have the same email address.
- How should the application enforce email uniqueness?
  The application should check whether the email already exists before creating a new account. Email addresses should be converted into lowercase before comparison so that different capitalization is treated as the same email.
- What could happen if two users register with the same email simultaneously?
  Both requests may check the database at the same time and find that the email is available. As a result, both may create accounts with the same email address. This can lead to duplicate accounts and login problems.
- What additional protection should be provided by the database?
  The database should have a UNIQUE constraint on the normalized email column. This ensures that duplicate email addresses cannot be stored, even when multiple registration requests happen simultaneously.
- One advantage and one disadvantage of using a strongly typed verification model:
  Advantage: It prevents invalid verification states and makes the code easier to understand and maintain. Disadvantage: It requires creating additional classes and takes more time to develop.